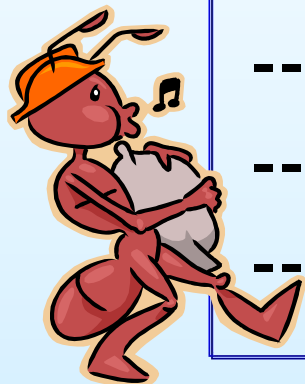


第4章：系统概要设计— 1.工程过程&设计原则

• 学习目的和要求



---系统概要设计的任务—**掌握**

---概要设计工程过程—**理解**

---软件设计的原则—**掌握**

□ 概要设计引言

设计阶段，系统要解决的问题是：系统 “怎么做”。

问题结构（软件需求）^{映射} → 软件结构

从软件需求规格说明书出发，
形成软件的具体设计方案。

对于较大规模的软件项目，软件设计往往被分为两个阶段进行：

- 概要设计（系统设计/总体设计）：确定软件系统的基本框架；
- 详细设计（部件级设计）：确定软件系统的内部细节。

□ 概要设计引言

➤ 从建筑设计看软件设计

设计良好的建筑应该展示出坚固、适用和令人赏心悦目的特点。

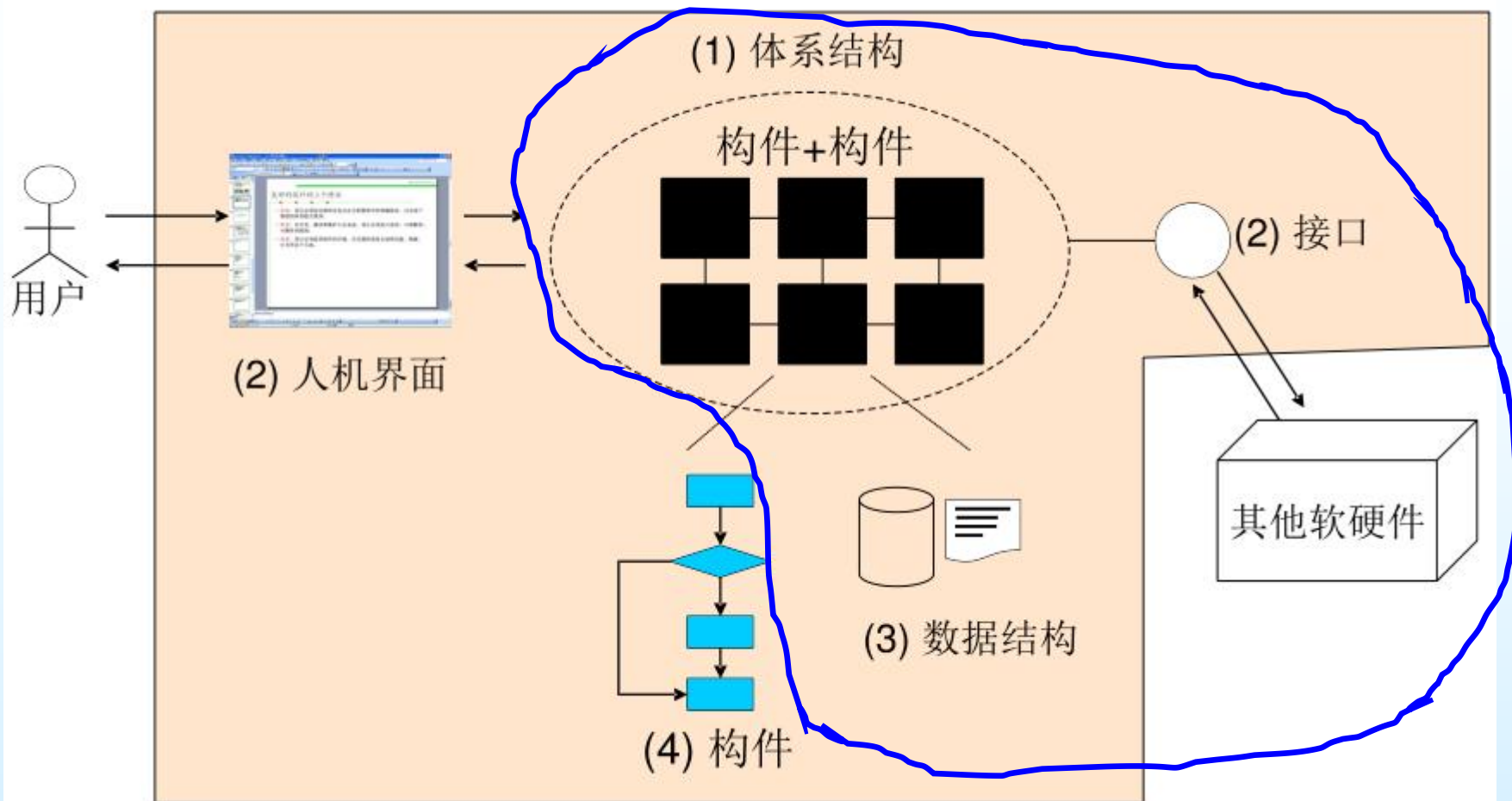


➤ 对好的软件设计来说也是如此：

- ✓ 适用：软件要符合开发的目标，满足用户需求；
- ✓ 坚固：软件应该不含任何妨碍其功能的缺陷；
- ✓ 赏心悦目：使用软件的体验应该是愉快的。

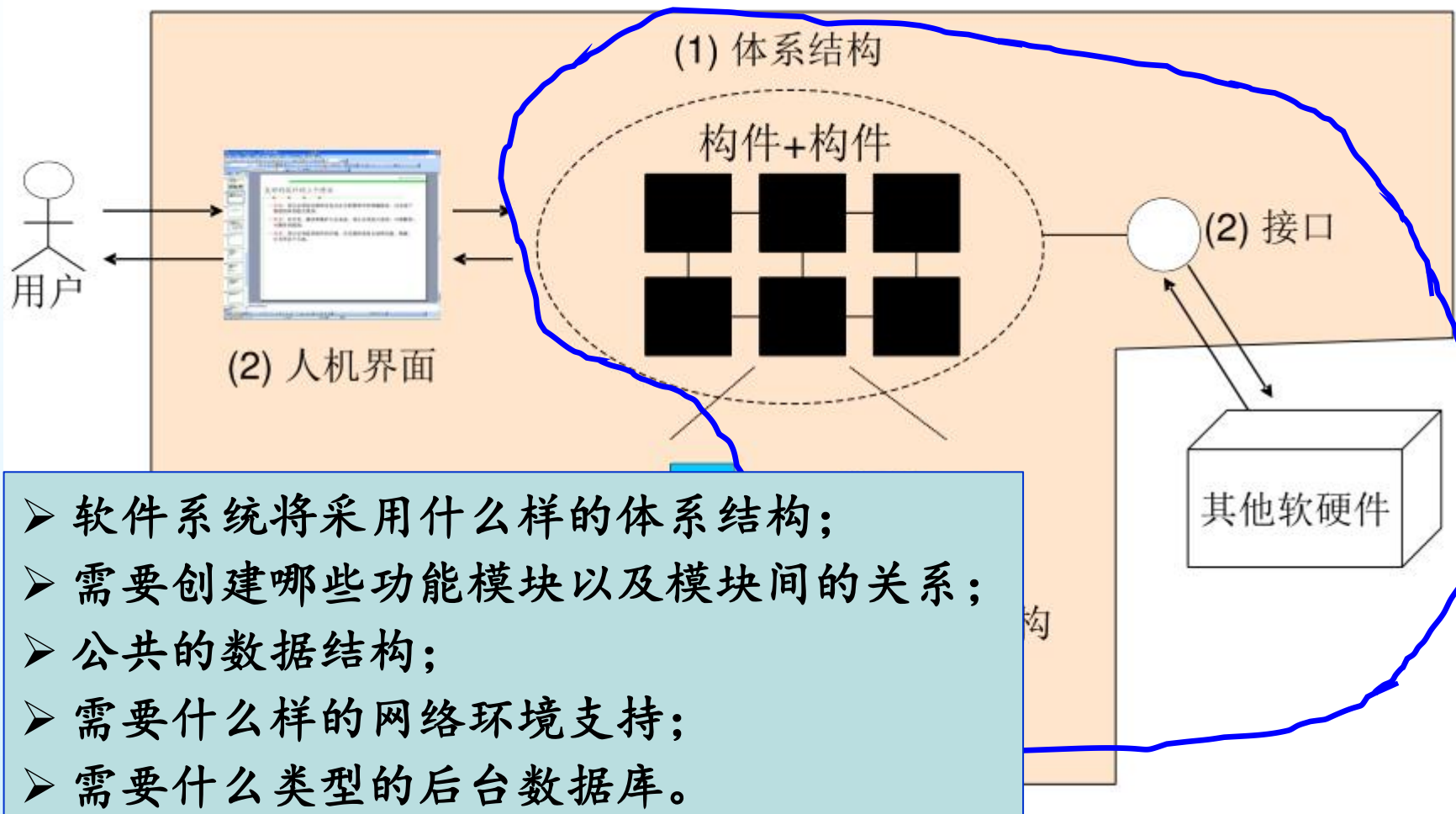
一、概要设计的任务

1、设计的元素



一、概要设计的任务

1、设计的元素



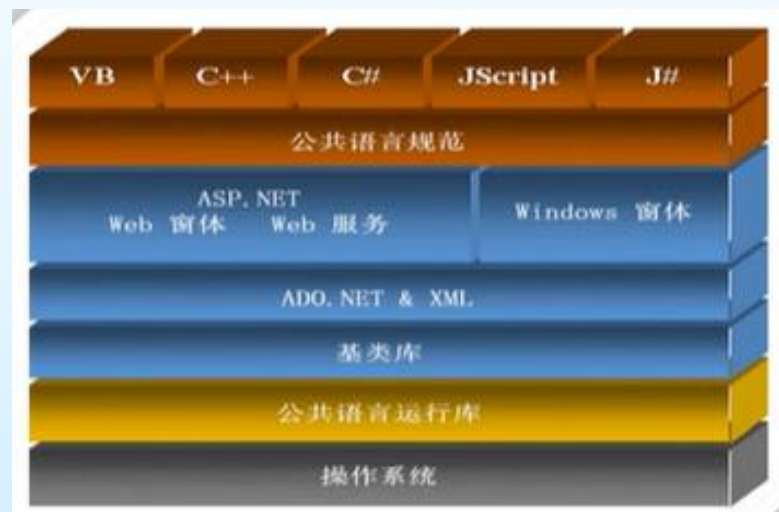
一、概要设计的任务

2、概要设计的内容

(1) 软件体系结构设计

选择适合于需求的软件体系结构风格，开展软件物理架构、逻辑架构设计。

如何以最佳的方式划分一个系统，如何标识组件，组件之间如何通信，信息如何沟通，系统的元素如何能够独立地进化。



一、概要设计的任务

2、概要设计的内容

(2) 接口设计

接口是类、构件或其他类（包括子系统）的外部可见的操作说明，而没有内部结构的规格说明。

- 与其他系统、硬件的外部接口
- 各种构件间的内部接口

例: 列检系统与零件仓库管理系统交互接口

	说明
交互机制	异步请求，系统调用零件仓库管理系统的接口用于备料，在完成备料之后调用本接口
接口技术	RestfulAPI，采用Java开发
访问地址	http://localhost:8081/material
参数类型	String
输入	零件Id、零件坐标
输出	返回提示信息

二、概要设计工程过程

概要设计主要包括三个方面的设计：一是系统物理架构设计，二是软件结构设计（接口）、三是数据结构设计，具体任务可分为9个活动：

1、制定规范：需要制定的标准和规范包括

- 设计文档编制标准；
- 信息编码格式，硬件、操作系统的接口规约；
- 设计目标、设计原则。

2、系统体系结构设计—物理架构设计

- 将系统分解为具有独立任务的子系统；
- 分析子系统之间的通信、确定子系统的外部接口
- 确定系统的软、硬件环境、网络环境和数据环境
- 对系统进行整体物理部署与优化设计

二、概要设计工程过程

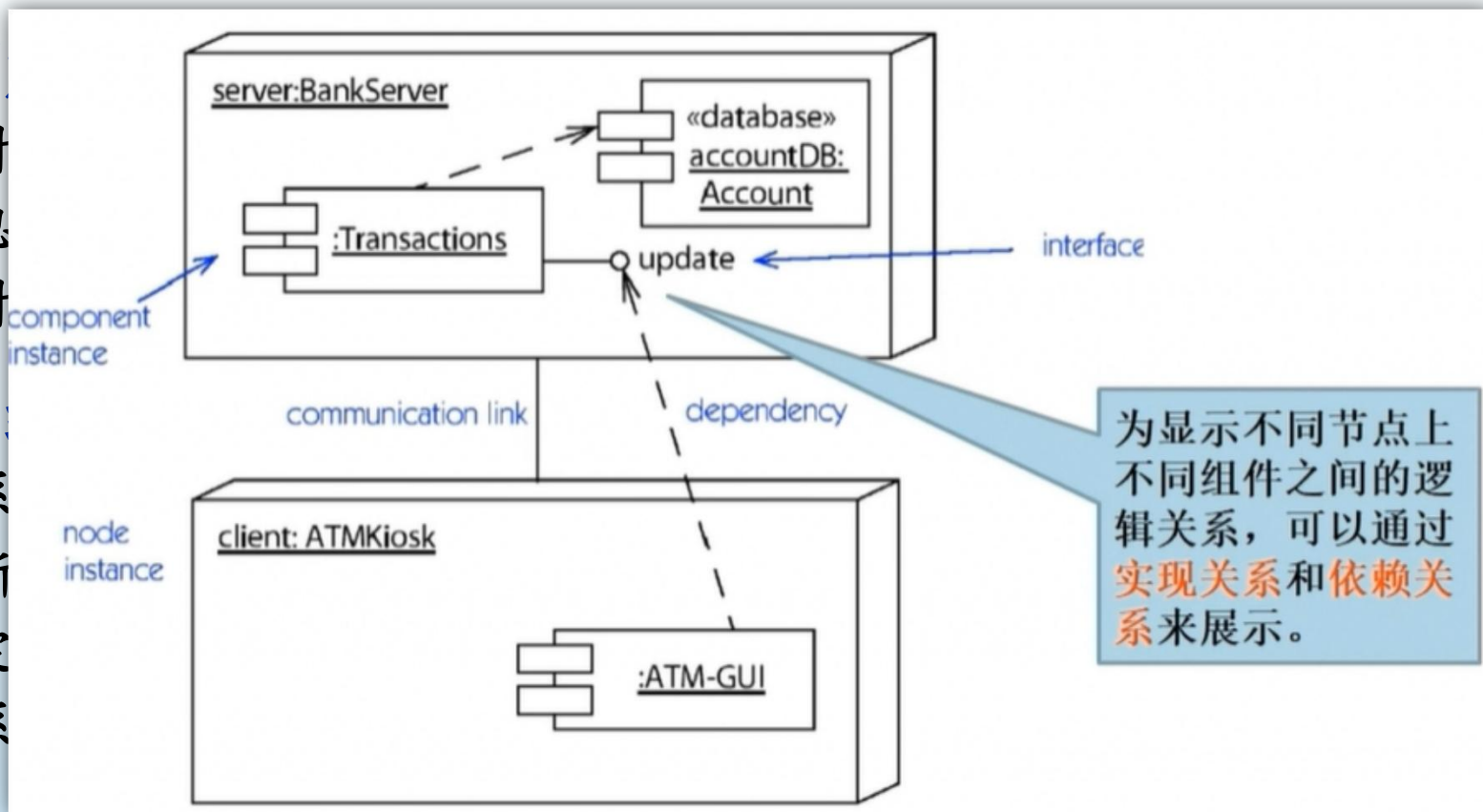
概要设计主要包括三个方面的设计：一是系统物理架构设计，二是软件结构设计（接口）、三是数据结构设计，具体任务可分为9个活动：

1、制

- 设计
- 信息
- 设计

2、系

- 将系
- 分析
- 确定
- 对系



二、概要设计工程过程

3、软件体系结构设计—逻辑结构设计

- 确定构造子系统的模块元素；
- 确定模块间的调用和返回关系
- 根据需求定义每个模块的功能；
- 评估软件质量，进行结构优化
- 定义模块接口及接口数据结构；

4、公共数据结构设计

- 公共数据变量的数据结构和作用范围
- 输入、输出文件的结构
- 数据库中的表结构、视图结构及数据完整性。

5、安全性设计

- 包括操作权限、操作日志管理设计、文件与数据加密设计及特定功能的校验设计等。

6、故障处理设计

- 对来自软件、硬件、网络的故障，进行备用设备、出错处理、数据备份等。

7、可维护性设计

- 在系统中设置用于系统监测的专用模块、预计今后进行功能扩充的模块等。

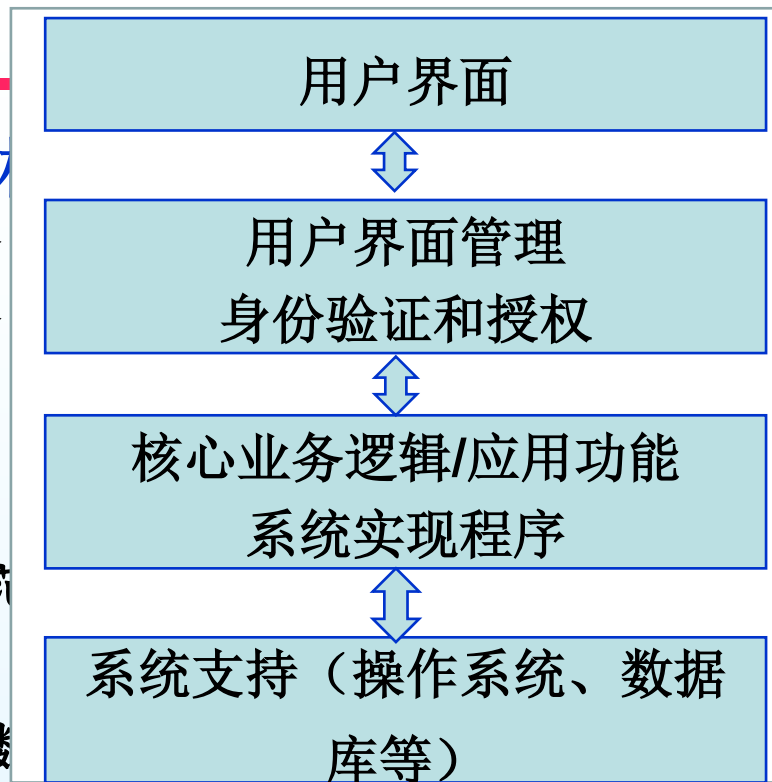
二、概要设计工程过程

3、软件体系结构设计—逻辑结构

- 确定构造子系统的模块元素；
- 根据需求定义每个模块的功能；
- 定义模块接口及接口数据结构；

4、公共数据结构设计

- 公共数据变量的数据结构和作用范围
- 输入、输出文件的结构
- 数据库中的表结构、视图结构及数



5、安全性设计

- 包括操作权限、操作日志管理设计、文件与数据加密设计及特定功能的校验设计等。

6、故障处理设计

- 对来自软件、硬件、网络的故障，进行备用设备、出错处理、数据备份等。

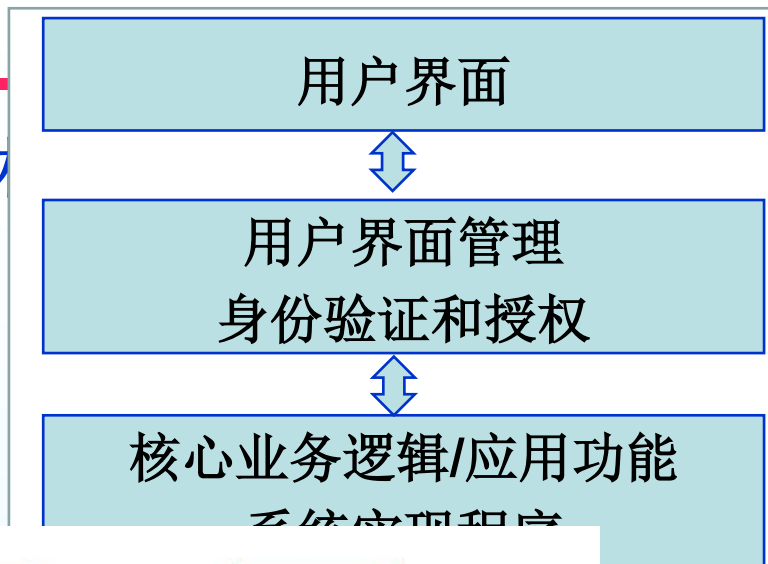
7、可维护性设计

- 在系统中设置用于系统监测的专用模块、预计今后进行功能扩充的模块等。

二、概要设计工程过程

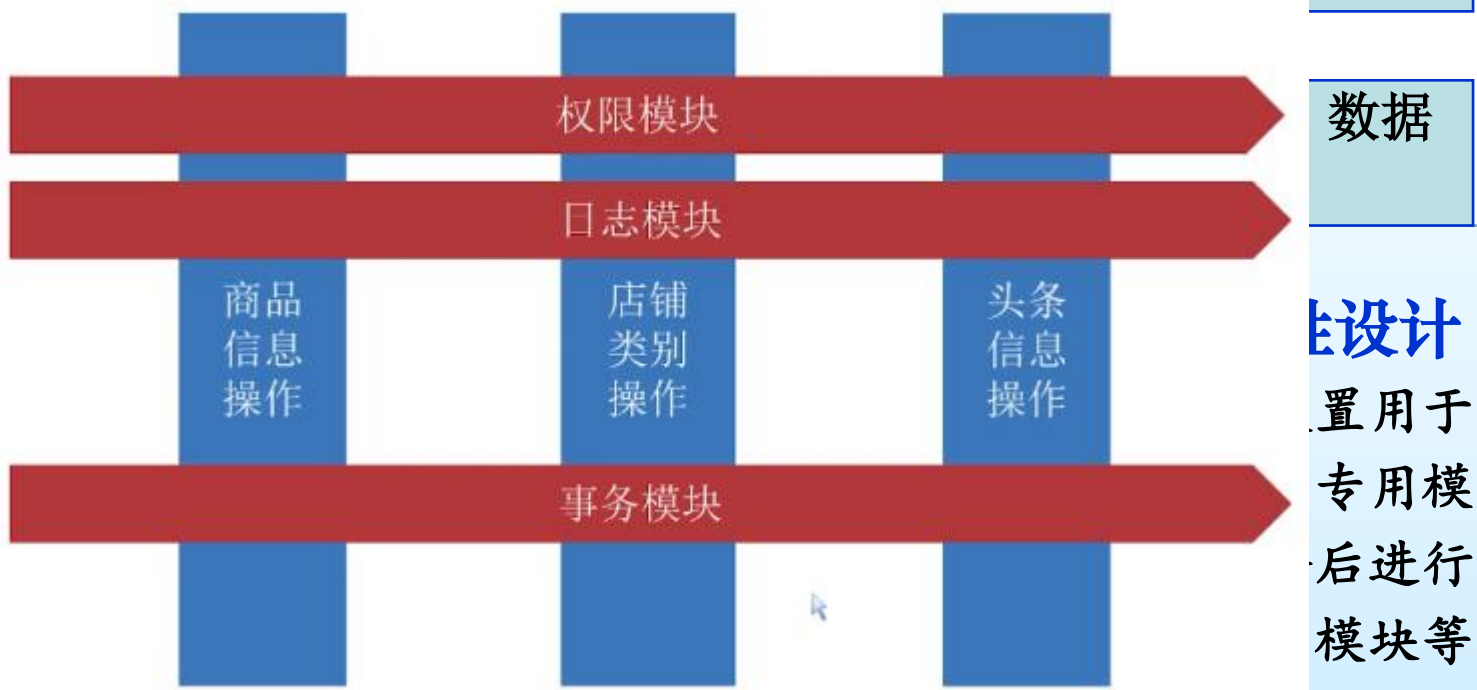
3、软件体系结构设计—逻辑结构

- 确定构造子系统的模块元素；
- 根据需求定义每个模块的功能；
- 定义模块接口及接口数据结构；



4、公共数据结构设计

- 公共
- 输入
- 数据



5、安全性设计

- 包括操作
- 日志管理
- 与数据
- 定功能

设计

置用于
专用模
后进行
模块等。

二、概要设计工程过程

8、编写概要设计报告

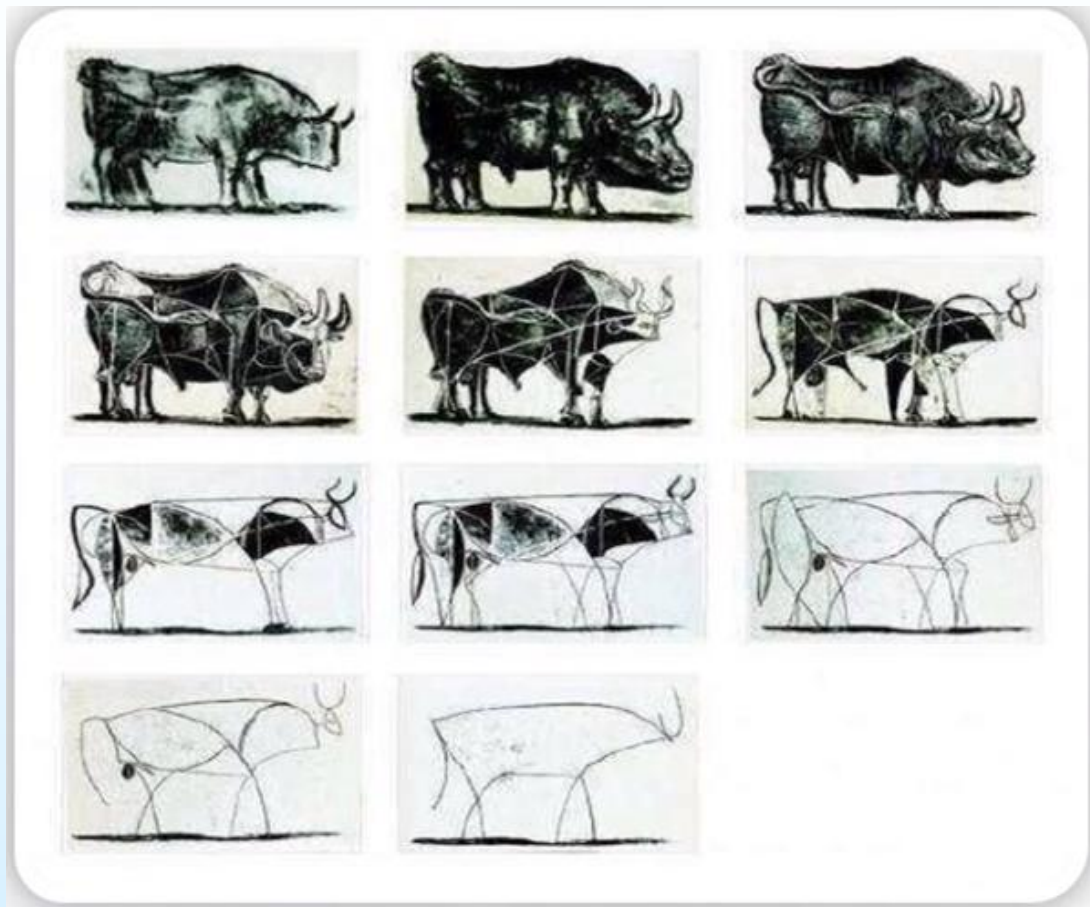
- 概要设计说明书
- 数据库设计说明书
- 测试初步计划

9、概要设计评审

- 需求确认：确认所设计的软件已覆盖了所有的软件需求。
- 接口确认：确认该软件的内部接口和外部接口是否明确定义。
- 模块确认：确认所设计的模块满足高内聚、低耦合，模块作用范围在控制范围内。
- 风险性：该设计在现有技术条件和预算下是否可按时实现。
- 可维护性：该设计是否考虑了今后的维护。
- 质量：该设计是否表现了良好的质量特征。

三、软件设计原则

1、抽象与逐步求精



(1) 抽象

□ 基本策略

在软件设计的规模逐渐增加的情况下，控制复杂性。

□ 过程

- 从特殊到一般。
- 上层概念是下层概念的抽象。
- 下层概念是上层概念的精化和细化。

三、软件设计原则

1、抽象与逐步求精

□ 软件设计的抽象包括过程抽象和数据抽象。

➤ 过程抽象（也称功能抽象）

任何一个完成明确定义功能的操作都可以被当作单个实体对待。

➤ 数据抽象

定义数据类型和施加于该类型对象的操作，并限定了对象的取值范围。

```
public class GeometricObject
{
    public String color;
    public Boolean filled;
    public String getColor()
    { return color; }
    public boolean isFilled ()
    { return filled; }
}
```

数据抽象

过程抽象

三、软件设计原则

1、抽象与逐步求精

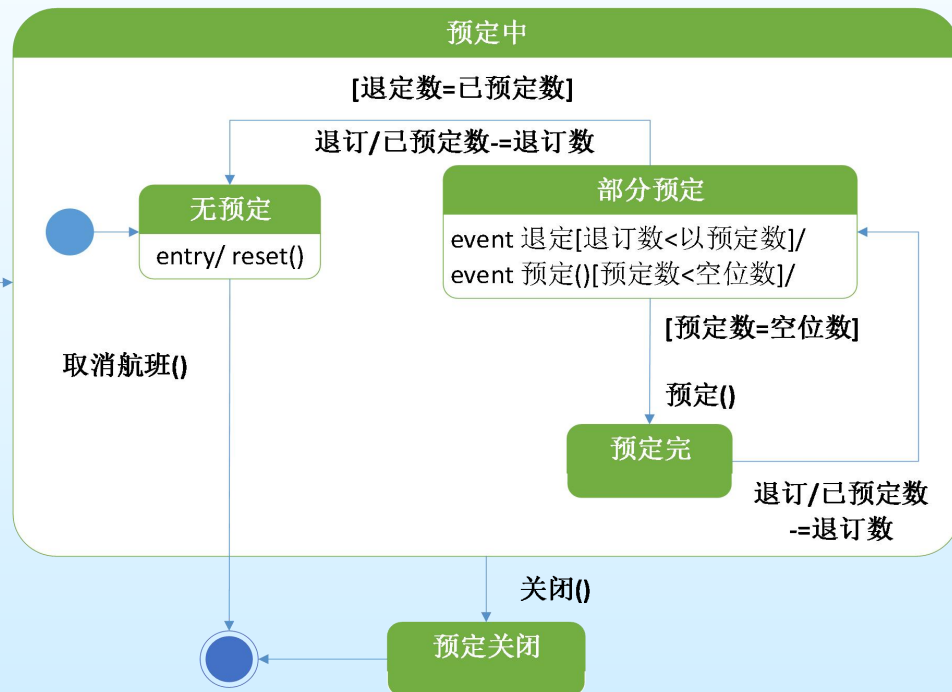
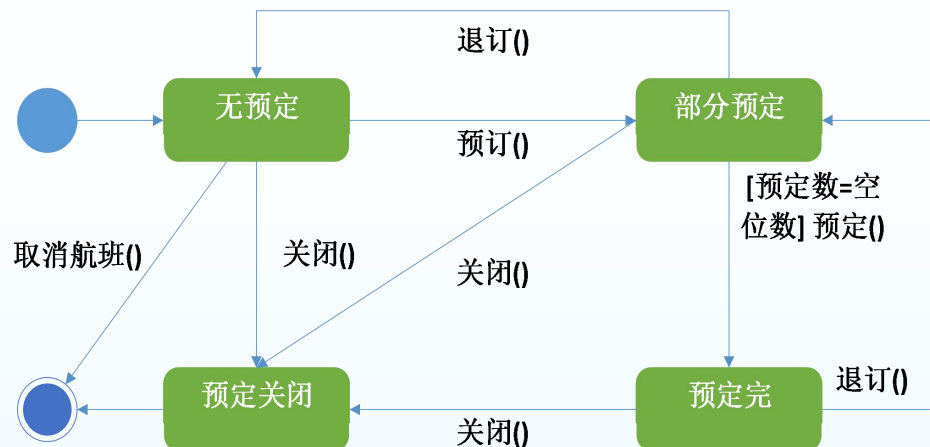
□ 逐步求精

- 把问题的求解过程分解成若干步骤或阶段
- 每步都比上步更精化，更接近问题的解法

□ 抽象与逐步求精的关系

- 抽象能够描述过程和数
- 据而忽略低层的细节。
- 求精有助于在设计过程中揭示低层的细节。

➤ 软件工程过程的每一阶段都是对软件解法的一次求精。



三、软件设计原则

2、模块化

软件体系结构是软件模块（构件）之间关系的表示，它决定了整个系统的结构，也确定了系统的质量。什么是模块？

□ 模块（构件）的概念

模块是数据说明、可执行语句等程序对象的集合，它是单独命名的，并且可以通过名字来访问。

例如：过程、函数、子程序、类、构件等。

□ 模块化

- 把软件按照规定原则，划分为一个个较小的，相互独立的但又相互关联的部件。
- 体现了系统分解和抽象的过程。

三、软件设计原则

2、模块化

- 设 $C(X)$ 是描述问题 X 复杂性的函数， $E(X)$ 是解决问题 X 所需工作量（按时间计算）的函数。对于两个问题 $p1$ 和 $p2$ ，如果 $C(p1) > C(p2)$ ，则 $E(p1) > E(p2)$ ，即问题越复杂，解决问题所需的花费更多。
- 通过对人解决问题的实验，又存在另一个有趣的规律： $C(p1+p2) > C(p1) + C(p2)$ ，即多个问题组合后的复杂性比单独考虑每个问题时的复杂性要大。
- 综合以上条件，可以得出： $E(p1+p2) > E(p1) + E(p2)$
- **结论（模块化的论据）：**将复杂问题分解成可以管理的片段会使解决问题更加容易。

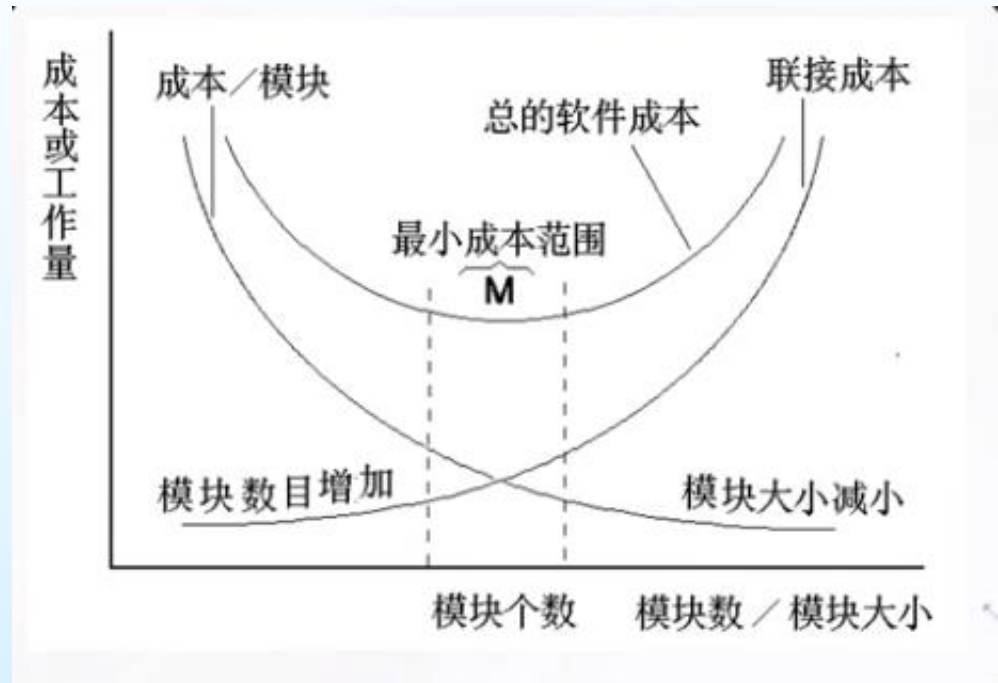
三、软件设计原则

2、模块化

□ 如果无限制地划分软件，开发它所需要的工作量会变得小到可以忽略吗？

□ 模块化的注意事项

- 如果模块是相互独立的，当模块变得越小，每个模块花费的工作量越低。
- 当模块数增加时，模块间的联系也随之增加，把这些模块联接起来的工作量也随之增加。



三、软件设计原则

3、信息隐藏

□ 信息隐藏的含义

- 每个模块的实现细节对于其它模块来说应该是隐蔽的。
- 模块中所包含的信息（包括数据和过程）不允许其它不需要这些信息的模块使用。

- 通过信息隐蔽，可以定义和实施对模块的过程细节和局部数据结构的存取限制。

```
public class GeometricObject
{
    public String color;
    public Boolean filled;
    private String getColor()
    { return color; }
    private void SetColor(String color)
    { this.color=color; }
}
```


三、软件设计原则

1.信息专家原则

3、信息隐藏

商品类: **Item**

流水号:201309140001		2013/09/14 16:46	
Products name	price	数量	金额
歌达利圆好吃点	10.00	1	10.00
016			
06902083883492	32.00	1	32.00
			2.00
			5.04
			7.04
			0.00

小票类: **Sale**

流水类: **SaleLineItem**

```
Class Sale {  
    List<SalesLineItem> lines;  
    Double GetTotal() {  
        Double total=0.0;  
        for(SalesLineItem line : lines) {  
            Item item=line.getItem();  
            Double quantity=line.getQuantity();  
            Double subTotal=item.getPrice()*quantity;  
            total+=subTotal; }  
        return total; }  
}
```

Sale类越级操作了

SalesLineItem类的商品,使得如购买奖励打折等活动时会存在**SalesLineItem**和**Sale**类的级联修改的问题

三、软件设计原则

1.信息专家原则

3、信息隐藏

```
public class Sale {  
    private List<SalesLineItem> lines;  
    private Double GetTotal() {  
    private Double total=0.0;  
        for(SalesLineItem line : Lines) {  
            total+=line.getSubTotal(); }  
        total= total * //商品组合优惠代码切入  
        return total; }  
}
```

SaleLineItem类只操作Item类的汇总

```
public Class SalesLineItem {  
    Item item;  
    Double Quantity;  
    Double getSubTotal() {  
        double subTotal = item.getPrice() * Quantity;  
        subTotal= subTotal * //商品数量优惠代码写入  
        return subTotal; } }  
}
```

Sale类只操作SaleLineItem类的汇总，没有越级操作Item类

- ❑ 将职责（方法）分配给具有实现该职责所需全部信息的直接类
- ❑ 在职责的具体实现中，尽量避免**越级操作**。

三、软件设计原则

2.创建者原则

3、信息隐藏

问题描述：收银机完成结算，形成一张小票，说明**Sale**和**SaleLineItem**对象已经被创建。那么问题是：这两个对象应该在什么地方和什么时间创建合适？

//收银记账类

```
public class Register {  
    private Sale sale=null;  
    public void recordSale(quantity){  
        if(sale==null) sale=new Sale();  
        let lineItem = new SalesLineItem(quantity)  
        sale.addSalesLineItem(lineItem); }  
}
```

越级创建了
LineItem类实例

导致**Sales**私有成员
对外暴露

//销售小票类

```
public class Sale {  
    private List<SalesLineItem> lines;  
    public void addSalesLineItem(SalesLineItem lineItem)  
    { lines.add(lineItem) }  
}
```

3、信息隐藏

```
//收银记账类
public class Register {
    private Sale sale=null;
    public void recordSale(quantity){
        if(sale==null) sale=new Sale();
        sale.createLinItem(quantity); }
}
```

LinItem实例由Sale类创建，保证了外部类无法控制LinItem实例，

创建者原则： 设类A和类B，满足以下任一条件，均可使用A类创建B类实例：

- A类组合B类（B是A的成员，B实例不能离开A实例单独存在）
- A类聚合B类（B是A的成员，B实例可以单独存在）
- A类记录B类（B不是A的成员，但是A的某个集合成员的元素）
- A类紧密使用B类（B不是A的成员，但A的某些方法紧密使用B）

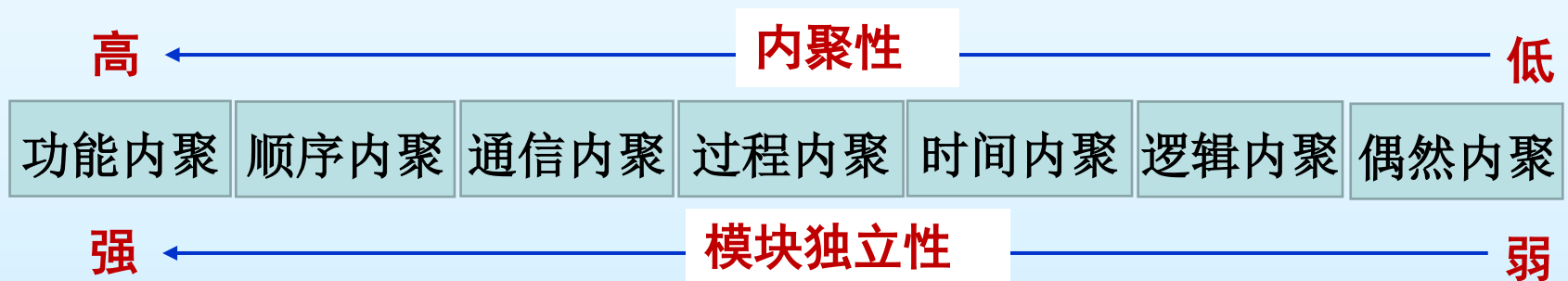
三、软件设计原则

4、功能独立性

功能独立性是指每个模块只完成系统要求的独立的子功能，并且与其他模块的联系最少且接口简单。

□ 功能独立性的衡量指标：内聚度与耦合度

内聚（cohesion）是一个模块内部各个元素彼此结合的紧密程度的度量。又称块内联系，若一个模块内各元素联系越紧密，它的内聚性越高，内聚性有几种类型：



三、软件设计原则

4、功能独立性

1) 偶然内聚： 指一个模块内各处理元素之间没有任何联系，这样的模块不易理解也不易修改，是最差的内聚情况。

2) 逻辑内聚： 指模块内执行几个逻辑上相似的功能，通过参数确定该模块完成哪一个功能。各部分之间在功能上有相互关系，但不易修改，当某个调用模块要求修改此模块公用代码时，另一些模块又不要求修改。调用此模块时不用的部分也占据内存，降低系统效率。

3) 时间内聚： 把需要同时执行的动作组合在一起形成的模块为时间聚模块。如初始化一组变量，同时打开若干文件等。时间内聚比逻辑内聚程度高一些，但这样的模块和其他模块耦合度较高。

4) 过程内聚： 如果一个模块内的处理是相关的，而且必须以特定次序执行，则称为过程内聚。它相对时间内聚的程度更强一些，但由于仅仅为完整功能的一部分，所以内聚度仍比较低。

三、软件设计原则

4、功能独立性

5) 通信内聚：指模块内所有处理元素都在同一个数据结构上操作（有时称为信息内聚），或指各处理时使用相同的输入数据或产生相同的输出数据。如一个模块完成：生产日报表、周报表、月报表，都使用同一数据即日产量。通信内聚的内聚性要高于前种类型。

6) 顺序内聚：指一个模块种各个处理元素都密切相关于同一功能且必须顺序执行，前一功能元素的输出是下一功能元素的输入。

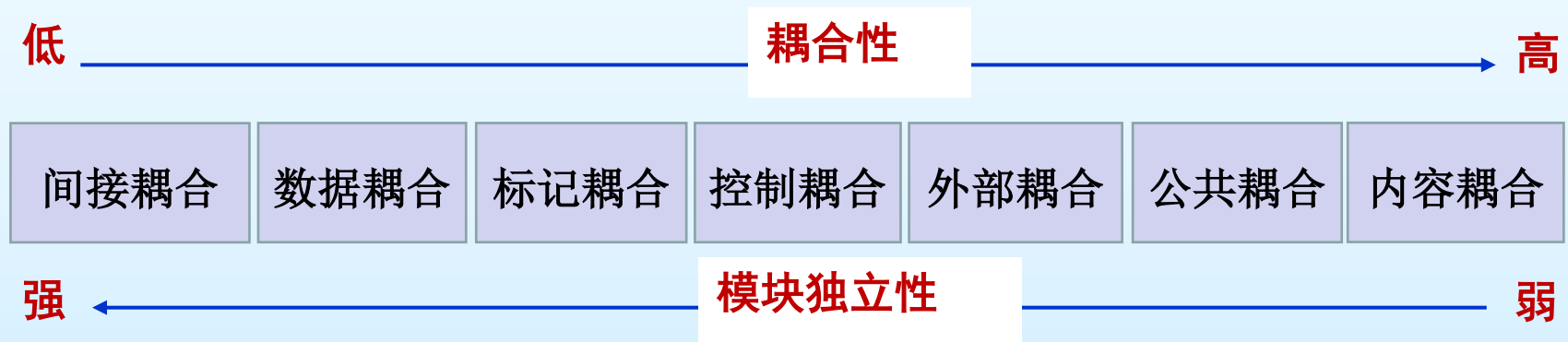
7) 功能内聚：是最强的内聚，指模块内所有元素共同完成一个功能，缺一不可。因此模块不能再分割。功能内聚的模块最易理解、易修改，它的功能是明确的单一的，与其他模块的耦合是弱的，功能内聚的模块有利于实现软件的重用。

三、软件设计原则

4、功能独立性

- 耦合是模块之间的相对独立性（互相连接紧密程度）的度量。耦合一般指代码块、类、模块、系统间的相互调用或引用。

模块间联系越紧密，耦合性越强，模块的独立性则越差。模块间的耦合高低取决于模块间接口的复杂性、调用的方式以及传递的信息。模块的耦合性又以下几种类型：



三、软件设计原则

4、功能独立性

1) 间接耦合：两个模块之间没有直接关系，它们之间的联系完全通过主模块的控制和调用来间接实现的。耦合度最弱。

```
//销售小票类
public class Sale {
    //流水信息
    private List<SalesLineItem> lines;
    //会员信息
    private Member member;
    //获取小票总金额
    public Double GetTotal() {
        Double total=0.0;
        for(SalesLineItem line : lines) {
            total+=line.getSubTotal(); }
        total= total *member.getDiscount();
        return total; }
}
```

间接耦合

三、软件设计原则

4、功能独立性

2) **数据耦合**：两个模块之间有调用/引用关系，传递的是数据值，相当于值模型传递。

```
//销售小票类
public class Sale {
    //流水信息
    private List<SalesLineItem> lines;
    //会员信息
    private Member member;
    //打印机
    private Printer printer
    public void print() {
        printer.print(member.getID(),member.getCredits(),...);
    }
}
```

数据耦合

三、软件设计原则

4、功能独立性

3) 对象耦合：两个模块之间有调用/引用关系，传递的是数据对象，相当于引用模型传递。

//销售小票类

```
public class Sale {
```

//流水信息

```
private List<SalesLineItem> lines;
```

//会员信息

```
private Member member;
```

//打印机

```
private Printer printer
```

```
public void print() {
```

```
    printer.print(member,lines); }
```

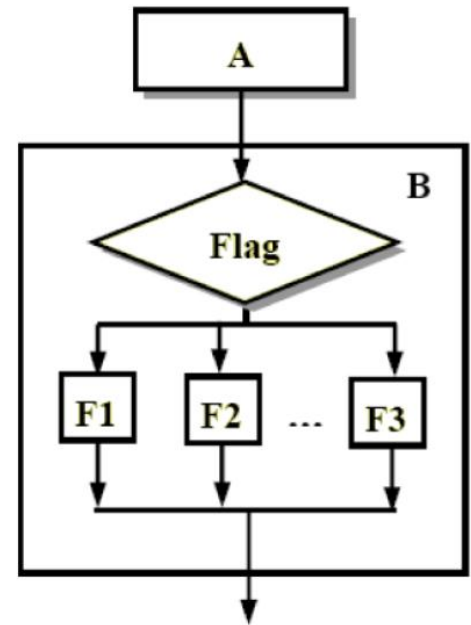
```
}
```

对象耦合

三、软件设计原则

4、功能独立性

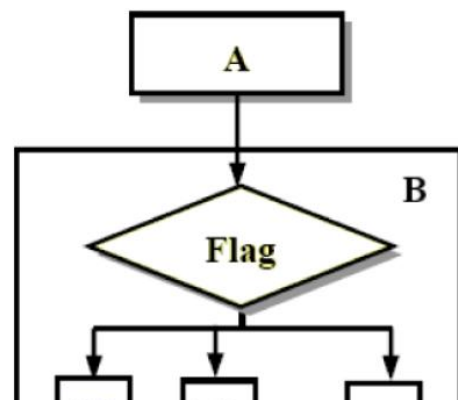
4) 控制耦合：如果模块传递的信息中有控制信息就称作控制耦合，如一个模块调用另一个模块时，传递的是控制变量（如开关、标志灯），被调用模块通过该控制变量的值有选择地执行块内某一功能，这种耦合实质上是在单一接口上选择多功能模块中的某项功能。因此对被控制模块的任何修改，都会影响控制模块。控制耦合属于中等程度的耦合，它增加了系统的复杂程度。



三、软件设计原则

4、功能独立性

4) 控制耦合：如果模块传递的信息中有控制信息就称作控制耦合，如一个模块调用另一个模块时，传递的是控制变量（如开关、标志灯），被调用模块通过该控制变量的值有选择地执行块



```
taxMaker.calculate(taxpayer.getFlag());
```

```
public class TaxMaker{  
    public double calculate(int flag) {  
        if(flag==0) { return tax*0.05 }  
        else if(flag==1) { return tax*0.07 }  
        else {return -1.0 }  
    }  
}
```

Taxplayer对象向TaxMaker对象传入了控制变量flag，使得Taxplayer和TaxMaker产生了控制耦合

此代码中，当未来出现新的税率变化时，这种耦合将导致相关其他程序模块也随之进行修改，产生级联效应。

三、软件设计原则

4、功能独立性

5) 外部耦合：如果一组模块都访问同一全局简单变量而不是同一全局数据结构，而且不是通过参数表传递该全局变量的信息，则称为外部耦合。

6) 公共耦合：指通过一个公共数据环境相互作用的那些模块间的耦合。公共数据环境可以是全局变量或数据结构，共享的通信区、内存的公共覆盖区及存储介质上的文件、物理设备等。

公共耦合复杂程度随耦合模块的个数增加而增加：

- 如某个模块有错，通过公共区将错误延伸到其他模块，影响软件的可靠性。
- 使软件的可维护性变差，若一模块修改了公共区数据，会影响到与此有关的所有模块。

三、软件设计原则

4、功能独立性

5) 外部耦合：如果一组模块都访问同一全局简单变量而不是同一全局数据结构，而且不是通过参数表传递该全局变量的信息，则称为外部耦合。

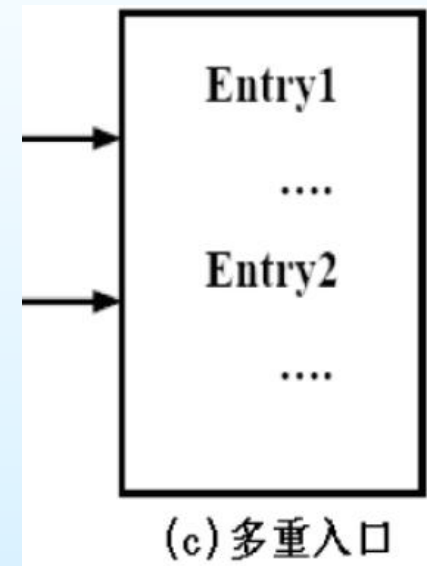
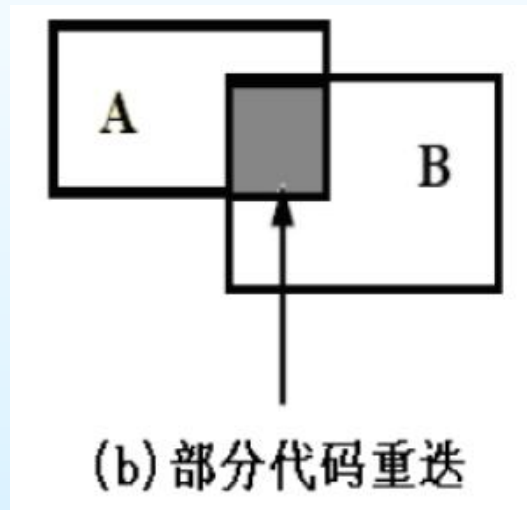
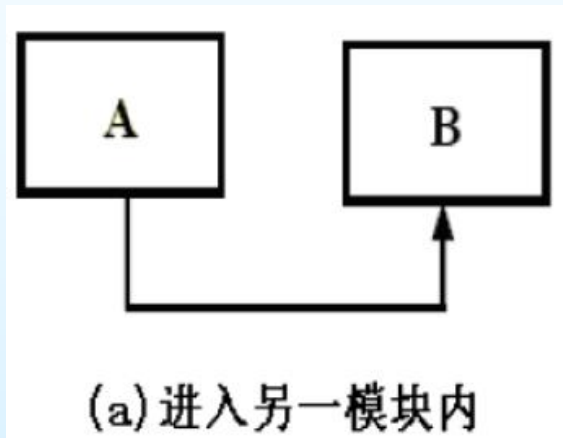
6) 公共耦合：指通过一个公共数据环境相互作用的那些模块间的耦合。公共数据环境可以是全局变量或数据结构，并含的通信口

```
public class SalesLineItem {  
    ItemDao dao=ItemDao.getDAO();  
    Item item=dao.getone(itemID);  
}  
  
public class DaySumItem {  
    ItemDao dao=ItemDao.getDAO();  
    Item item=dao.getone(itemID);  
}
```

三、软件设计原则

4、功能独立性

7) 内容耦合：是最高程度的耦合，也是最差的耦合。内容耦合的模块常具有以下特征：一个模块直接访问另一模块的内部数据，一个模块不通过正常入口转到另一模块的内部，一个模块有多个入口或者两个模块有部分代码重迭。



4、功能独立性

由于会员信息要求满足三级等保要求，其他模块不得直接访问会员系列类（普通会员、微信会员、银卡会员、金卡会员等）的方法。



- 与会员高度相关的类需要频繁调用会员类，以便实现会员的分类、存储、权益等。
- 外部的（进销存）模块也需要频繁访问会员信息，以便实现会员积分、储值消费等。

4、功能独立性

□ 解决方案之一

(1) 把普通会员、微信会员、银卡会员、金卡会员等类，形成继承链（内容耦合）。

(2) 每层继承的会员类的公共成员，设置为protected修饰，以便子类可以进行内部访问。

(3) 与会员类高度相关的其他类，应该与会员类置于同一个包中，通过default修饰的方法，进行包内访问。

(4) 外部类通过服务接口或远程方法（REST或Web Service）访问会员类的内容。

4、功能独立性

□ 解决方案之一

(1) 把普通会员、微信会员、银卡会员、金卡会员等类，形成继承链（内容耦合）。

(2) 每层继承的会员类的公共成员，设置为protected修饰，以便子类可以进行内部访问。

(3) 与会员类高度相关的其他类，应该与会员类置于同一个包。

高内聚低耦合原则总结：

(1) 具有高度相关功能的模块或类，可以采用相对较高耦合度的交互方式进行连接（如继承、对象引用）。

(2) 功能差异较大的模块或类，应该采用耦合度较低的交互方式进行连接（如接口调用，抽象类调用）

三、软件设计原则

5、控制器原则



收银机的UI界面按下结算键时，应该触发结算事件和执行相应的结算功能。

问题：接收该结算事件并负责进行处理的类应该怎样设计？

```
//创建一个结算按钮的UI类，在类中实现结算功能
public class SettleButton extends JButton {
    public SettleButton() {
        this.setText("结算");
        this.setSize(120,70);
    }
    public void actionPerformed(event e) {
        //执行结算代码
    }
}
```

三、软件设计原则

5、控制器原则



收银机的UI界面按下结算键时，应该触发结算事件和执行相应的结算功能。

问题：接收该结算事件并负责进行处理类应该怎样设计？

```
//创建一个结算按钮的UI类，在类中实现结算功能
public class SettleButton extends JButton {
    public SettleButton() {
        this.setText("结算");
        this.setSize(120,70);
    }
}
```

问题：在实践中，收银机每个键位常常根据业务需求，调整每个按钮的具体功能和快捷键位。上述代码直接将功能以代码写死到了UI控件中，后续将难以扩展。

三、软件设计原则

5、控制器原则

□ 解决思路

```
public class JButton {  
    //ActionListener是抽象类  
    private ActionListener listener;  
    //UI按钮触发按下事件时，调用ActionListener  
    public void buttonPressed(Event e) {  
        listener.actionPerformed(e);    }  
    public void addListener(ActionListener listener) {  
        this.listener=listener;    }  
}
```

定义功能控制器

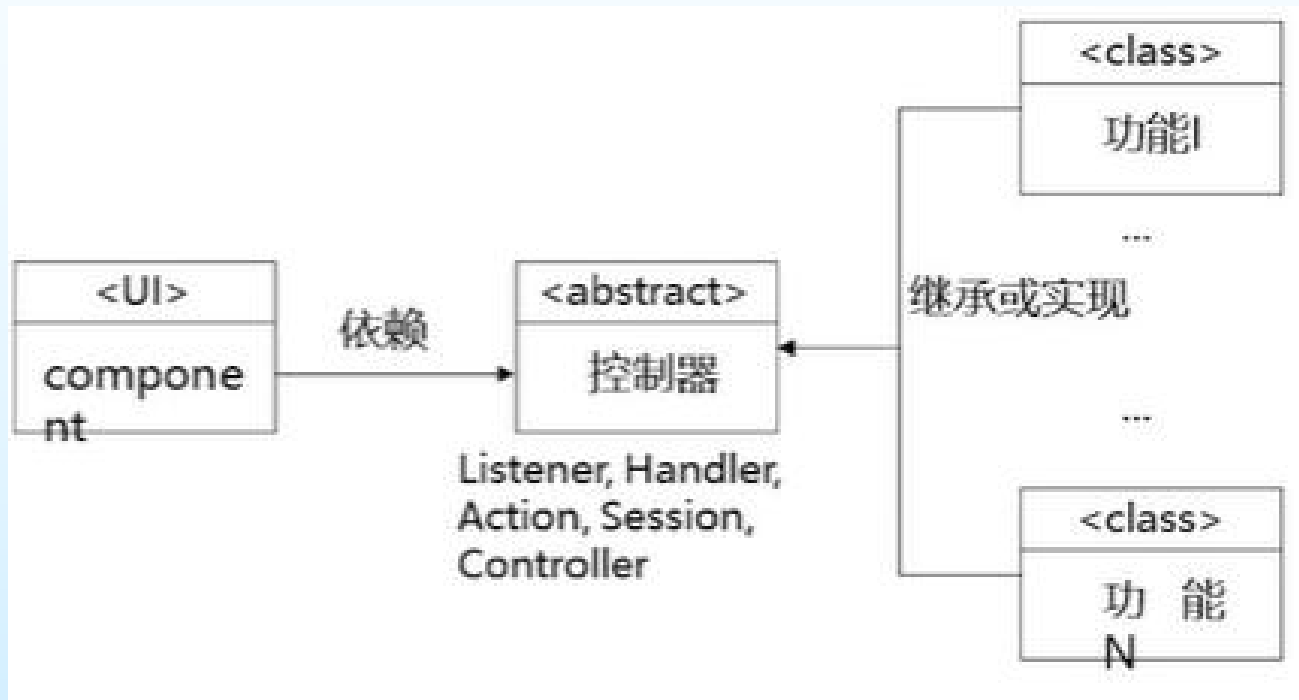
```
public class SettleListener implements ActionListener {  
    //实现这个约定好的接口方法  
    private void actionPerformed(Event e) {  
        //执行结算代码  
    }  
}
```


三、软件设计原则

5、控制器原则

控制器是**UI层**（或**Component**）之上的第一个对象，它负责接收和处理由**UI层**产生的事件。

在各类软件架构中，常见的命名是**Listener**(Java中)、**Handler**(C++、C#)、**Action**、**Session** (J2EE)、**Controller**(Springboot)等。



谢谢观看

